

Navigating the BLE Maze with the ESP32

A Lighthearted Tale of Triumph Over Tech Tangles

When you set out to pair an ESP32 as a Bluetooth Low Energy (BLE) client with an Arduino UNO R4 WiFi as a BLE server, you might expect a smooth ride. After all, BLE is the low-power cousin of classic Bluetooth, perfect for IoT projects like controlling an LED with simple "ON" and "OFF" commands. But, as we discovered, the path to BLE bliss is paved with quirks, conflicts, and a few facepalm moments. Why BLE and not classic Bluetooth? The UNO R4 WiFi's hardware supports BLE natively via its NINA-W102 module, making it the go-to choice for low-power communication in this setup. Classic Bluetooth, alas, wasn't an option without extra hardware. Buckle up as we recount the chronological chaos of getting the ESP32 BLE client to play nice with the UNO R4 server, complete with the problems we faced and the fixes that saved the day.

The BLE Adventure Begins: A Chronological Comedy of Errors

Problem 1: Library Wars – ArduinoBLE vs. ESP32 BLE

Our journey kicked off with a classic case of library conflict. The UNO R4 server relied on the `ArduinoBLE` library, while the ESP32 client needed the ESP32-specific BLE library (`BLEDevice`, `BLEUtils`, etc.). Compiling the ESP32 client code in the Arduino IDE was like trying to herd cats—one library would overwrite the other, leading to errors like class redefinitions or missing functions (e.g., `getScan`).

Resolution: We played library hide-and-seek. By isolating the ESP32 BLE library in a project-specific `lib` folder, we avoided clashes with the global `ArduinoBLE` library. For extra measure, we temporarily renamed `ArduinoBLE` to `ArduinoBLE_backup` during client compilation. In PlatformIO, we cleared the build cache to banish stale references and used conditional compilation directives to keep the peace. Result? The ESP32 client compiled without a hitch.

Problem 2: Compilation Conundrums

Next up, the ESP32 client code threw tantrums during compilation. Incorrect callback instantiations and wonky `connect` function calls triggered a cascade of errors, leaving us staring at cryptic logs in PlatformIO.

Resolution: We rolled up our sleeves and refactored the code. Reordering class definitions ensured callbacks were properly initialized, and we fixed the `connect` method to align with the ESP32 BLE library's API. A quick tweak to the `platformio.ini` file, specifying the

`featheresp32` board for our Adafruit ESP32 Feather (with ESP32-WROOM-32E), sealed the deal. The code compiled, and we high-fived the screen.

Problem 3: Watchdog Woes and Crashing Clients

Just when we thought we were in the clear, the ESP32 client started crashing during connection attempts, with the serial monitor ominously reporting watchdog timer (WDT) timeouts. It was as if the ESP32 was throwing a tantrum, refusing to talk to the UNO R4 server.

Resolution: We tamed the beast with a non-blocking connection approach, preventing the ESP32 from hanging during BLE operations. Adding aggressive WDT resets in the client code kept timeouts at bay, and we temporarily disabled the WDT for debugging to pinpoint the issue. Updating the ESP32 BLE library to the latest version squashed potential stack bugs, and the client finally stayed alive.

Problem 4: Connection Limbo

The ESP32 could see the UNO R4 server advertising, but connecting? That was another story. The client either crashed or hung, while the server's serial output showed it polling for connections like a lonely satellite.

Resolution: We went full detective mode. Adding verbose Serial print statements traced the connection process, revealing where it stalled. Testing the UNO R4 server with a BLE app (nRF Connect) confirmed it was advertising correctly, pointing the finger at the client. We ensured the UUIDs (`SERVICE_UUID: 19B10000-E8F2-537E-4F6C-D104768A1214`, `CHARACTERISTIC_UUID: 19B10001-E8F2-537E-4F6C-D104768A1214`) matched perfectly and adjusted the client's connection logic to be non-blocking. Heap tracing and memory checks ruled out resource issues, and soon, the client and server were chatting like old friends.

Problem 5: Serial Port Shenanigans on Mac

Uploading code to the ESP32 on a Mac turned into a mini-nightmare when the serial port (`/dev/cu.usbmodem34B7DA63129C2`) vanished, despite no hardware changes. The Arduino IDE and PlatformIO were equally baffled.

Resolution: We played tech support roulette. Resetting the USB port with `sudo kextunload` in Terminal, checking for rogue processes hogging the port, and verifying CH340/CH341 driver installations got us back on track. Testing in Safe Mode ruled out system conflicts, and specifying the correct port in `platformio.ini` ensured smooth uploads. The Mac and ESP32 made up, and uploads resumed.

Problem 6: Board Definition Doubts

We briefly suspected the `featheresp32` board definition in `platformio.ini` was causing trouble, given our Adafruit ESP32 Feather with ESP32-WROOM-32E.

Resolution: A quick check confirmed `featheresp32` was correct, but we tested `esp32dev` to rule out board-specific quirks. Tweaking `platformio.ini` to ensure proper framework and library settings kept everything copacetic.

Problem 7: Messages Lost in Transit

Finally, the ESP32 client connected but failed to send "ON" and "OFF" messages to toggle the UNO R4's LED. The server was listening, but the messages weren't arriving.

Resolution: We double-checked the client's `pRemoteCharacteristic->writeValue` method and added a 2-second delay between messages to match the project's requirements. On the server, we ensured the characteristic had write permissions and matched UUIDs. Serial print statements on both sides confirmed successful transmission, and the LED blinked happily on pin 13.

The Happy Ending

By May 22, 2025, we had a working setup: the ESP32 client connected to the UNO R4 WiFi server, sending "ON" and "OFF" every 2 seconds to toggle an LED. The `platformio.ini` files were streamlined, and READMEs documented the victory.

Recommendations for Your ESP32 BLE Journey

Ready to embark on your own ESP32 BLE adventure? Here's the game plan:

- **Use the Right Library:** Stick with the ESP32 BLE library (`BLEDevice`, `BLEUtils`, etc.) for the ESP32 client. It's tailored for Espressif's hardware and supports BLE operations like scanning and connecting. Avoid `ArduinoBLE` for the ESP32—it's designed for boards like the UNO R4 WiFi and will cause conflicts. If you're mixing boards (like we did), isolate libraries in project-specific folders or use PlatformIO's dependency management to keep things tidy.
- **Why ArduinoBLE Doesn't Work for ESP32:** The `ArduinoBLE` library is optimized for Arduino boards with specific BLE hardware (e.g., NINA-W102 on the UNO R4). It lacks the low-level control needed for the ESP32's BLE stack, leading to compilation and runtime errors.
- **Best Practices:**
 - **Non-Blocking Code:** Use non-blocking logic for BLE operations to avoid crashes and WDT timeouts.
 - **UUID Precision:** Ensure service and characteristic UUIDs match exactly between client and server.
 - **Debugging Tools:** Leverage Serial print statements and apps like nRF Connect to verify advertising and connections.
 - **PlatformIO for the Win:** Use PlatformIO for better library management and build control, especially in multi-board projects. Clear the cache if builds act funky.

- Mac Serial Fixes: Keep CH340/CH341 drivers updated and know your Terminal commands for USB port resets.
- Test Incrementally: Verify each step (scanning, connecting, writing) before adding complexity.
- Avoid Common Pitfalls: Don't mix BLE and classic Bluetooth libraries, and double-check board definitions in `platformio.ini`. If crashes persist, update the ESP32 BLE library and check memory usage.

With these tips, you'll dodge the BLE gremlins we wrestled. Happy coding, and may your ESP32 BLE projects shine as brightly as that LED on pin 13!

References

You can check out working code for the server [here](#), and for the client [here](#).